

Ejercicios y actividades resueltas

1) Ejercicios

Ejercicio 1: Análisis comparativo (RDBMS vs NoSQL)

Enunciado: Comparar un RDBMS (ej. MySQL) con un NoSQL (ej. MongoDB) considerando escalabilidad, rendimiento, costo, soporte y comunidad. Luego justificar cuál conviene para un sistema de comercio electrónico global.

Comparación rápida

Criterio	MySQL (RDBMS)	MongoDB (NoSQL)
Tipo de datos	Tablas con filas y columnas; relaciones claras	Documentos (JSON/BSON); estructura flexible
Consistencia	Muy buena para transacciones (ventas, pagos)	Buena, pero depende del diseño; suele priorizar flexibilidad
Escalabilidad	Escala bien, pero muchas veces se siente más “natural” crecer con réplicas y ajustes; el salto a partición (sharding) puede ser más delicado	Escala de forma práctica con sharding y clústeres, sobre todo en lecturas y grandes volúmenes
Rendimiento	Excelente para operaciones estructuradas y consultas con joins bien diseñadas	Muy bueno para lecturas rápidas y datos semiestructurados; joins no son su fuerte
Costo	Puede ser bajo (open source) + costo de operación	Puede ser bajo (community) o subir según servicio/cluster; operación también cuenta
Soporte y comunidad	Amplia comunidad; muchos recursos y experiencia en el mercado	Gran comunidad; muy usado en apps modernas y servicios en nube

¿Qué conviene para un e-commerce global?

Para un **comercio electrónico global**, lo más común es usar **los dos** con roles diferentes:

- **MySQL (o PostgreSQL)** para lo crítico y transaccional: pedidos, pagos, facturación, inventario, devoluciones (aquí la consistencia es clave).
- **MongoDB** para partes más flexibles o que crecen rápido: catálogo de productos con atributos variables, recomendaciones, historial de navegación, logs.

Si debo escoger solo uno:

- Elegiría un **RDBMS** (MySQL/PostgreSQL) como base principal, porque un e-commerce necesita mucha seguridad en transacciones.

Ejercicio 2: Diseño de arquitectura (gestión hospitalaria)

Enunciado: Proponer una arquitectura para un sistema de gestión hospitalaria que integre DBMS relacional y no relacional. Explicar ventajas de combinar ambos y justificar las decisiones.

Propuesta (en simple)

Objetivo: que el hospital registre pacientes, citas, diagnósticos y pagos sin perder datos, y que además guarde documentos (imágenes, PDFs) sin complicarse.

Arquitectura sugerida (3 capas):

1. **Cliente (web o app):** pantallas para admisión, médicos, farmacia, caja.
2. **Servidor de aplicación (API):** aquí viven las reglas (roles, permisos, validaciones).
3. **Capa de datos (dos bases de datos):**
 - **Relacional (PostgreSQL / SQL Server / MySQL):**
 - Pacientes, médicos, citas, recetas, pagos, stock de farmacia.

- Reglas de integridad (por ejemplo, no se puede cobrar una cita que no existe).
- **NoSQL (MongoDB):**
 - Historias clínicas con campos variables.
 - Archivos y metadatos: exámenes, imágenes, notas, documentos (guardando la ruta o referencia si el archivo está en un almacenamiento aparte).

Ventajas de combinar

- **Orden y control** en lo importante (relacional).
- **Flexibilidad** para información médica que puede cambiar de formato (NoSQL).
- El sistema queda más fácil de ampliar sin romper lo que ya funciona.

Justificación

- Lo que tiene “números y reglas” (cobros, citas, inventario) va mejor en relacional.
- Lo que puede variar mucho (notas médicas, formatos, adjuntos) se maneja mejor en NoSQL.

Ejercicio 3: Proyecto con Big Data (GPS de vehículos)

Enunciado: Simular un caso donde una empresa de transporte analiza millones de registros GPS. Proponer un DBMS compatible con Big Data e integrar con una plataforma de análisis en la nube.

Caso

La empresa recibe coordenadas cada 5 o 10 segundos por vehículo. En un mes, eso se vuelve un volumen enorme.

Propuesta de solución

Parte A: Almacenamiento y consultas rápidas

- Guardar eventos (GPS) en un sistema pensado para volumen:
 - Opción 1 (muy usada): **Data Lake** en la nube (por ejemplo, almacenamiento tipo “bucket”) + formato columna (Parquet).
 - Opción 2: una base distribuida para eventos (según lo permitido en el curso): **Cassandra** (si se trabaja por series de tiempo) o un motor similar.

Parte B: Base relacional para lo “estable”

- Un RDBMS (PostgreSQL/SQL Server) para:
 - Vehículos, conductores, rutas, clientes, permisos.

Parte C: Análisis (Big Data)

- Usar una plataforma de procesamiento:
 - **Spark** para procesar grandes volúmenes.
 - Consultas analíticas con un motor tipo **warehouse** (según lo disponible): por ejemplo, un servicio analítico en la nube.

¿Qué se puede analizar?

- Rutas más usadas.
- Tiempos de parada.
- Velocidad promedio.
- Alertas: “salida de ruta”, exceso de velocidad.

2) Actividades

Evidencia principal

Cuadro comparativo y análisis técnico de DBMS: SQL Server, MySQL, PostgreSQL, Oracle y MongoDB.

Producto esperado

1. **Cuadro comparativo estructurado.**
2. **Informe corto** que explique en qué caso conviene cada DBMS.

2.1 Cuadro comparativo (resumen práctico)

DBMS	Tipo	Modelo de datos	Seguridad (idea general)	Rendimiento (idea general)	Escalabilidad (idea general)	Compatibilidad con arquitecturas	Ventajas	Limitaciones
SQL Server	Relacional	Tablas, relaciones, SQL	Fuerte (roles, auditoría, integración empresarial)	Muy bueno en sistemas empresariales	Escala bien; suele crecer fuerte en vertical y con réplicas/servicios	Centralizada, cliente-servidor (3 capas), y escenarios distribuidos (según configuración)	Integración con herramientas empresariales; reporting muy sólido	Licenciamiento puede ser costoso; requiere buena administración

MySQL	Relacional	Tablas, relaciones, SQL	Buena (depende de configuración)	Muy bueno para apps web	Buena; se apoya en réplicas y diseño correcto	Centralizada y cliente-servidor; puede usarse en distribuida con replicación/partición	Popular, mucha documentación y comunidad	Para escenarios muy grandes, requiere diseño cuidadoso
PostgreSQL	Relacional	Tablas, relaciones, SQL	Fuerte (roles, extensiones, buen control)	Excelente para consultas complejas y datos consistentes	Muy buena; soporta replicación y particiones (según diseño)	Centralizada, cliente-servidor y distribuida (con replicación y herramientas)	Robusto, muy usado en instituciones	Operación y afinamiento requieren práctica
Oracle	Relacional	Tablas, relaciones, SQL	Muy fuerte (entornos críticos)	Muy alto en escenarios grandes	Muy alta (según licencias y arquitectura)	Cliente-servidor, distribuida y nube en entornos corporativos	Estándar en bancos/telecom grandes; herramientas completas	Alto costo y complejidad de administración
MongoDB	No relacional	Documentos (JSON/BSON)	Buena; depende del diseño y controles	Muy bueno en lecturas y datos flexibles	Muy buena con clúster y sharding	Cliente-servidor y distribuida de forma natural (clústeres)	Flexible; ideal para catálogos, logs, perfiles	No es lo más cómodo para transacciones complejas con muchas relaciones

2.2 Informe corto (pertinencia por escenarios)

Escenario A: Sistema de gestión académica (universidad)

Recomendación: PostgreSQL o SQL Server.

- Motivo: se requiere consistencia (matrículas, notas, pagos), controles, reportes.

Escenario B: Tienda virtual mediana (web)

Recomendación: MySQL o PostgreSQL.

- Motivo: transacciones claras; buen rendimiento; fácil encontrar soporte.

Escenario C: Empresa grande con procesos críticos (banco, telecom)

Recomendación: Oracle o SQL Server (enterprise).

- Motivo: alta seguridad, auditoría, soporte corporativo, alta disponibilidad.

Escenario D: App que cambia seguido (catálogo con muchos atributos, redes, contenidos)

Recomendación: MongoDB.

- Motivo: la estructura del dato puede cambiar sin “romper” tablas.

Conclusión

- Para procesos con reglas y transacciones (pagos, matrícula, inventario), conviene un **relacional**.
- Para datos flexibles, grandes volúmenes de eventos o estructuras variables, conviene **NoSQL**.
- En proyectos grandes, es común usar un enfoque **mixto**: relacional + NoSQL + herramientas de análisis.